Universidade do Sul de Santa Catarina - UNISUL
Unidade Curricular: **Gestão e Qualidade de Software**
Professor:
Jorge Werner

A3 – Desempenho de compreensão

Meta de compreensão relacionada: **Aplicar o processo de gestão de configuração de software e criar repositórios em ferramentas de controle de versão.**

1. Contexto da atividade

Com a crescente demanda por sistemas mais robustos e a transformação digital acelerada, as empresas estão cada vez mais voltadas para a adoção de metodologias ágeis e práticas de DevOps. Nesse cenário, a integração contínua (*Continuous Integration*) e a entrega contínua (*Continuous Delivery*) têm se tornado padrões essenciais para garantir que o software seja entregue com qualidade, segurança e de forma rápida ao mercado. Essas práticas permitem que as equipes de desenvolvimento identifiquem rapidamente erros e inconsistências no código, reduzindo o tempo de correção e aprimorando a eficiência do processo de desenvolvimento.

Além disso, a automação de testes tem ganhado destaque como uma das principais formas de garantir a qualidade do software em ciclos de desenvolvimento rápidos e iterativos. Com a automação, é possível executar uma bateria completa de testes sempre que houver uma mudança no código, garantindo que novas funcionalidades não quebrem as já existentes e que o software se mantenha estável ao longo do tempo. A implementação de pipelines de CI/CD (*Continuous Integration/Continuous Deployment*) automatizados e a utilização de contêineres, como Docker, também são práticas que têm sido amplamente adotadas pelas empresas para melhorar a qualidade e a eficiência do desenvolvimento de software.

A transformação digital também trouxe um foco maior na experiência do usuário e na necessidade de softwares que não só atendam aos requisitos funcionais, mas que também sejam seguros, confiáveis e de fácil manutenção. O aumento das regulamentações, como o GDPR (*General Data Protection Regulation*), exige que as empresas garantam que seus softwares estejam em conformidade com as leis de proteção de dados, tornando a qualidade do processo de desenvolvimento uma prioridade estratégica. Neste contexto, a habilidade de criar e gerenciar um plano de testes robusto se torna ainda mais crucial.

Por fim, com o advento de práticas como o "*shift-left testing*", onde os testes são realizados o mais cedo possível no ciclo de desenvolvimento, as empresas estão mudando a mentalidade de que testes são uma fase final do processo. Ao invés disso, a qualidade do software está sendo integrada em todas as fases do desenvolvimento, desde o planejamento até a entrega. Isso não só melhora a qualidade do produto final, mas também promove uma cultura de responsabilidade compartilhada pela qualidade entre todos os membros da equipe, do desenvolvedor ao especialista em DevOps.

2. Objetivos

As equipes serão formadas por cinco estudantes. Qualquer composição diferente deverá ser combinada previamente com o professor. A escolha dos grupos é de responsabilidade dos alunos. Cada equipe desenvolverá um sistema do zero a partir de uma das temáticas abaixo. O professor definirá a distribuição para evitar duplicidade ou realizará sorteio, conforme o tamanho da turma.
Temáticas propostas, adaptadas do exemplo de projeto: Sistema de Cadastro de Academia; Sistema de Reserva de Hotéis; Sistema de Gestão de Lanchonetes; Sistema de Biblioteca; Sistema de Cadastro de Alunos; Sistema de Gestão de Estoque.

. O docente irá sortear e informar a temática para os grupos. Cada equipe deverá construir uma aplicação executável (web, desktop, móvel ou API) com linguagem e ferramentas acordadas com o professor. Recomenda-se começar com um fluxo principal e poucas regras de negócio, ampliando o sistema somente quando a base estiver funcionando.
Na primeira entrega, a equipe deve informar: problema atendido; usuários ou atores; tecnologias previstas; três ou mais requisitos funcionais; requisitos de qualidade prioritários; e um fluxo principal com critérios de aceitação. Todos os projetos devem incluir pelo menos um cadastro ou registro, uma consulta e uma regra de negócio validada por testes. Persistência em banco de dados é recomendada quando compatível com o escopo e com as competências trabalhadas na UC.

2.1 Objetivos específicos:

1) Compreender e aplicar práticas de qualidade de software:
    a. Os alunos deverão aplicar conceitos de qualidade de software, como integração contínua, automação de testes, e estratégias de testes (unitário, integração, validação, sistema).
2) Desenvolver habilidades de planejamento e execução de testes:
    a. Os alunos deverão elaborar um plano de testes completo que cubra todos os níveis do sistema, garantindo a cobertura dos principais requisitos funcionais e não funcionais.
3) Desenvolver competências em gestão de configuração e versionamento de software:
    a. Utilizando ferramentas de controle de versão, os alunos deverão gerenciar as modificações no código-fonte, garantindo a rastreabilidade e a gestão de configuração do projeto.
4) Promover a capacidade de trabalho em equipe e comunicação:
    a. Os alunos deverão trabalhar em equipes, dividindo as responsabilidades e comunicando de forma eficaz os progressos e desafios encontrados durante o desenvolvimento do projeto.

3. Descrição das atividades

1) Formação de Equipes:
    a. Os alunos serão divididos em equipes de 5 membros, com papéis definidos (líder de equipe, desenvolvedor, testador, especialista em DevOps). Mas todos devem serem programadores para desenvolver os testes unitários.
2) Análise do Sistema:
    a. As equipes deverão realizar uma análise detalhada de algum sistema, identificando os pontos críticos de melhoria e os principais requisitos de qualidade que precisam ser atendidos. A linguagem de programação e o ambiente deve ser comum aos membros do grupo.
3) Elaboração do Plano de Testes:
    a. Com base na análise, a equipe deverá elaborar um plano de testes completo, incluindo testes de unidade, integração, validação e sistema. O plano deve incluir a estratégia de testes, ferramentas utilizadas, e o cronograma de execução.
4) Implementação e Execução dos Testes:
    a. A equipe deve implementar os testes planejados utilizando ferramentas de automação de testes e integração contínua. A execução dos testes deverá ser contínua ao longo do semestre, com ajustes conforme necessário. A cobertura deve ser de 75% do código fonte.
5) Gestão de Configuração:
    a. As equipes deverão gerenciar as mudanças no código e na infraestrutura do projeto utilizando um sistema de controle de versão Github, documentando todas as alterações e garantindo a integridade do sistema.
6) Resolução de *Issues*
    a. O trabalho é um trabalho contínuo ao longo do semestre, portanto os professores podem lançar *issues* nos repositórios do trabalho. Estes *issues* precisam ser sanados até a entrega do trabalho. Estes *issues* se não resolvidos podem representar descontos no trabalho.
7) Relatório Final e Apresentação:

a. Ao final do semestre, as equipes deverão entregar um relatório detalhado (trabalho escrito) com o plano de testes, os resultados obtidos, as lições aprendidas e uma apresentação para a turma e os professores.

4. Escrito e implementação (7,0 pontos)

Devem constar do **trabalho escrito** pelo menos os seguintes tópicos: (0,5 pontos)
1) Atender as normas do caderno de metodologia da UNISUL modelo TCC, disponível em:
https://www.unisul.br/sou-estudante/biblioteca/
2) Devendo incluir no **mínimo**: Capa, Sobrecapa, Sumário, Introdução, Desenvolvimento, Conclusão e Referências.
3) Devem constar no escrito:

- 1 Introdução
- 2 Desenvolvimento (Plano de teste)
  - 2.1. Objetivo do plano de testes (0,20 pontos)
    - Descrição clara dos objetivos do plano de testes.
    - Justificativa para a execução dos testes no sistema legado.
  - 2.2. Escopo do Teste (0,2 pontos)
    - Definição do que será testado (módulos, funcionalidades, integrações).
    - Identificação do que não será testado e justificativa.
  - 2.3. Contexto do Sistema (0,20 pontos)
    - Descrição do sistema legado, incluindo sua arquitetura, tecnologias utilizadas e dependências.
    - Histórico de mudanças e manutenções realizadas.
    - Documentação existente (se disponível).
  - 2.4. Estratégia de Testes (0,20 pontos)
    - Definição dos tipos de testes a serem realizados:
      - Testes unitários
      - Testes de integração.
      - Testes de sistema.
      - Testes de regressão (se necessário).
      - Testes de desempenho (se necessário).
      - Testes de segurança (se aplicável).
    - Abordagem para lidar com a falta de documentação ou conhecimento do sistema.
  - 2.5. Ambiente de Teste (0,20 pontos)
    - Descrição do ambiente onde os testes serão executados.
    - Requisitos de hardware, software e configurações necessárias.
    - Estratégia para replicar o ambiente, se necessário.
  - 2.6. Dados de Teste (0,20 pontos)
    - Definição de como os dados de teste serão obtidos ou gerados.
    - Estratégia para garantir que os dados sejam representativos do ambiente de produção.
    - Tratamento de dados sensíveis ou confidenciais.
  - 2.7. Ferramentas de Teste (0,20 pontos)
    - Lista de ferramentas que serão utilizadas para automação, gestão de testes, monitoramento, etc.
    - Justificativa para a escolha das ferramentas.

- Desenvolver a automação dos testes unitários ao ser realizado um novo deploy (exemplo Github actions).
- Realizar a análise estática do código fonte (exemplo Integração com Maven e Sonarcloud).

- 2.8. Cronograma (0,20 pontos)
  - Estimativa de tempo para cada fase de testes.
  - Dependências e marcos importantes.

- 2.9. Responsabilidades (0,20 pontos)
  - Papéis e responsabilidades de cada membro da equipe. Definir os papéis para os como líder de equipe, desenvolvedor, testador e especialista em DevOps. Todos devem ser programadores para desenvolver os testes unitários.
  - Distribuição das tarefas entre os membros da equipe. Criar uma tabela com 3 colunas. A primeira coluna com o nome do aluno, a segunda coluna com a funcionalidade e a terceira com link(s) do(s) arquivo(s) com o teste unitário da funcionalidade.

- 2.10. **Critérios de Aceitação (4,0 pontos)**
  - Definição dos critérios que devem ser atendidos para considerar os testes bem-sucedidos. Deve ser obter pelo menos **75% de cobertura do código**.
  - Passar no *quality gate*:
  - Resolver todos *issues*.

- 2.11. Gestão de Defeitos (0,20 pontos)
  - Processo para reportar, rastrear e resolver defeitos encontrados durante os testes.
  - Ferramentas utilizadas para gestão de defeitos.

- 2.12. Relatórios e Métricas. (0,3 ponto)
  - Indicar as funcionalidades e os respectivos testes unitários.
  - Indicar onde foi aplicado a análise estática do código fonte com relação a medidas de: segurança, confiabilidade, manutebilidade, duplicações e complexidade até conseguir passar pelo portão da qualidade (*quality gate*). Registro e análise da solução para os problemas de qualidade encontrados no código fonte inicialmente e sua posterior correção.

- 2.13. Controle de versão (0,20 pontos)
  - Link do projeto no Github
  - Usar branch **main**.
  - Estrutura do repositório (licença, readme, src)
  - Convenção das mensagens de commits.

- 3 Conclusão

Atenção: A avaliação da distribuição das tarefas de desenvolvimento (código fonte) de forma igualitária entre os membros do grupo será realizada através dos commits dos integrantes do grupo no seu repositório. Commits de arquivos que não sejam código fonte (ex.: Readme, pom.xml, e outros) não serão considerados. Cada integrante deve utilizar sua conta pessoal no GitHub para realizar as alterações. Somente ou autor deve realizar o commit dos seus arquivos. Somente será considerado o código fontes da última versão do programa no repositório. Os commits em arquivos que não existentes na última versão do software não serão considerados. Pontos serão atribuídos ou removidos de cada aluno de acordo com os itens da implementação. É de responsabilidade do grupo acompanhar e registrar todas as alterações no repositório através de commit. Um histórico de commits saudável deve ser organizado, compreensível, atômico e rastreável, permitindo que qualquer pessoa — inclusive o próprio desenvolvedor no futuro — entenda claramente o que mudou, por que mudou e quando mudou, sem precisar adivinhar. Para isso, é essencial que apresente alterações frequentes e granulares, evitando "megacommits" que concentrem diversas modificações não relacionadas em um único registro. Cada commit deve ter uma finalidade bem definida, como corrigir um bug no login ou adicionar um teste unitário para uma função específica, pois essa abordagem mantém o repositório organizado, facilita a compreensão da evolução do código e possibilita a reversão de mudanças problemáticas de forma simples e segura.

4. Apresentação (3,0 pontos)

T**odas as equipes** devem **realizar a apresentação do trabalho. Todos** os integrantes devem falar sobre o trabalho e participar da apresentação do trabalho. **Deve** ser utilizado algum software de apresentação tipo PowerPoint. Todos os grupos têm o tempo médio é de 10 minutos (variação de 2 minutos) para a apresentação.

No mínimo devem constar na apresentação os seguintes tópicos:

- Capa
- Sumário
- Requisitos funcionais e não funcionais
- Demonstração das funcionalidades com o software em execução
    - Integração Contínua
    - Automação dos Testes unitários
- Principais *issues* detectados e corrigidos
- Evolução do software (antes e depois do *quality gate*)
- Conclusão

Importante: A apresentação deve **iniciar** pela identificação de todos os alunos nome, semestre e curso (0,5 pontos). Como é um trabalho em grupo **todos** os integrantes devem apresentar parte do conteúdo, portanto dividam o tempo de forma igualitária entre os membros. Cada integrante deve demonstrar parte da execução do software (1,5 ponto). A apresentação deve seguir uma sequência lógica e coerente no tempo estabelecido (1,0 pontos).

Observação: A socialização poderá ser revertida na exposição dos trabalhos na Mostra da A3 ou Expotech a critério do professor.
Os trabalhos estão incluídos também no Anima HUB (way.animahub.com.br) e devem cumprir todos os requisitos no edital específico, replicados para lá.

5. Artefatos a serem entregues

Todos os alunos devem fazer a entrega dos artefatos do trabalho. Caso algum aluno não entregue, será descontado **0,5 pontos** por não fizer a postagem no prazo e local especificado (ulife). Postagem de arquivos não solicitados equivale a não postar o trabalho. **Após o prazo final não é possível fazer o envio ou correções.**

Os artefatos a serem entregues são.

- Link do repositório do github
- Arquivo do trabalho (PDF).

- Arquivo com slides da Apresentação (PDF).

Compacte os arquivos a serem enviados em um arquivo .zip ou .rar para realizar uma única postagem. Os alunos e o grupo são **responsáveis** pela correta postagem do arquivo da A3 no sistema.

6. Datas

- Dia **05/10/2026** o trabalho da A3 será apresentado e solicitado a turma.
- Até o dia **05/10/2026** o link do repositório do github e o nome dos integrantes da equipe deve ser postado **por todos os integrantes** no Ulife na seção conteúdo no item "A3 – Grupo".
- Até o dia **29/11/2026** o trabalho deve ser postado **por todos os integrantes** do grupo no Ulife na seção conteúdo no item "A3 - Postagem do trabalho".
- Socialização das apresentações entre os dias **30/11/2026.**

7. Acompanhamento

Os professores definirão, em momento oportuno durante as aulas, períodos destinados à orientação e ao acompanhamento do desenvolvimento do trabalho. Para tratar especificamente da A3, o agendamento deverá ser solicitado por e-mail com, no mínimo, 24 horas de antecedência. A presença de todos os integrantes do grupo é obrigatória.

8. Defesa de Trabalho

Se necessário o professor pode convocar a defesa do trabalho de forma individual para comprovação da elaboração do trabalho.

Boa Atividade!